

Session 04

Polymorphism

واحد من مبادئ ال OOP و اللي بيركز علي تعدد الاشكال

Poly --> Many

morphism --> Forms

Types of polymorphism

1- Static | Compile time | Early

2- Dynamic | Run-Time | Late

1- Overloading: This allows the developer to create multiple methods with the same name but different in behavior, the compiler knows the different between them by the order, number, and types of the parameters.

2- Overloading: Its is implemented with inheritance allowing the base class to change the bahvior of the Derived class

Overloading:

Types of overloading:

1- Constructor Overloading

2- Indexer Overloading

3- Method Overloading

4- Operator Overloading

Operator Overloading Syntax

```
public static return_type operator operator_Symbol(Type t)
{
    //Logic
}
// Where Type Must Be A class or struct.
```

Casting Operators Overloading

```
public static explicit|implicit operator operator(parameter)
{
    //Custome Implementaion
}
```

NOTE: when implementing implicit overloading it will be usable as explicit to, Its recommended to type explicitly as its more readable

```
#region Methods Overloading

public static int SumNumbers(int x , int y)
{
    return x + y;
}

public static double SumNumbers(int x , double y)
{
    return x + y;
}

public static double SumNumbers(double x , double y)
{
    return x + y;
}

public static int SumNumbers(int x , int y , int z)
{
    return x + y + z;
}

#endregion

#region Operator Overloading

public static Complex operator +(Complex left, Complex right)
{
    return new Complex()
    {
        Real = (left?.Real ?? 0) + (right?.Real ?? 0),
        Img = (left?.Img ?? 0) + (right?.Img ?? 0)
    }
}
```

```

    };
}

public static explicit operator int(Complex C)
{
    return C?.Real ?? 0;
}

#endregion

```

There are Matching Operators meaning You can overload an operator without its matching one or twin one

Ex: > and < Operators -> they both must be implemented if u are implementing one of them

Overriding

Overriding -->

What Is Overriding ?

Its Base Class object member Method that got inherited by a Derived Class allowing that class to change the behavior of that functions in two ways either by masking the old Function if its not virtual , or override it if it is Virtual.

```

public class TypeA
{
    public int A { get; set; }
    public TypeA(int a)
    {
        A = a;
    }
    // Non-Virtual Method
    public void MyFun01()
    {
        Console.WriteLine("This Is MyFun01 From Base And I Am
Base");
    }
    // Virtual Method
    public virtual void MyFun02()

```

```

        {
            Console.WriteLine($"This Is MyFun02 From Base And TypaA : A
= {A}");
        }
    }

    public class TypeB : TypeA
    {
        #region Properties

        public int B { get; set; }

        #endregion

        #region Constructors
        public TypeB(int a, int b) : base(a)
        {
            B = b;
        }

        #endregion

        #region Methods

        // This Is Masking or Hiding The Inherited Member
        TypeA.MyFun01() [New           Version - Masking]
        // For Ignoring The Compiler Warnings Use The "new" Keyword
        public new void MyFun01()
        {
            Console.WriteLine("This Is MyFun01 From Derived And I Am
Child");
        }

        // Using Keyword "Override" To Override Inheritance Member
        TypeA.MyFun02()
        // TypeA.MyFun02() Must Be Market As virtual Method and Non-
        Private in Its           First Appearance
        // Dynamic Polymorphism
        public override void MyFun02()
        {
            Console.WriteLine($"This Is MyFun02 From Derived"
            +" TypeB : A = {A}  , B = {B}");
        }
    }

```

```
    #endregion  
}
```

Note: The Different between both is more clear when working with Binding

Binding

Types OF Binding

1- Late Binding [Run-Time - Dynamic]: Happens In Run Time The Compiler Checks For The Derived Class Inherited overrided Method And Call it If the Child class version if it did not override it its look for other class in the hierarchy that override that method so It Happens Dynamically

2- Early Binding [Compile-Time - Static]: Happens During Compile Time Thats Why its Early and its happens with the inherited masked methods